zeroriscy_debug_unit

OVERVIEW

The Debug Unit is a critical support module in the Zero-Riscy RISC-V processor that enables external debugging, testing, and verification of processor operations. It provides a controlled interface between the processor core and an external debugger, allowing developers to monitor, pause, inspect, and modify processor behavior during program execution.

This module plays a major role in system validation, fault analysis, and software debugging by providing visibility into the processor's internal state.

It supports important debugging operations such as:

- Halting processor execution
- Resuming execution after halt
- Reading and writing registers
- Accessing Control and Status Registers (CSRs)
- Monitoring program execution flow
- Handling traps and exceptions during debug sessions

FUNCTIONAL DESCRIPTION: CORE RESPONSIBILITY

The primary function of the Debug Unit is to provide full control and observation of the processor while it is running or halted.

It acts as an interface between:

- The processor core
- The external debugging system

This allows safe and controlled debugging without affecting processor reliability.

The debug unit ensures that developers can:

- Pause execution at any stage
- Inspect register and CSR values
- Modify processor state when required
- Resume execution after debugging
- Analyze exceptions and trap conditions

This makes the module essential for processor development, verification, and post-silicon debugging.

1. CLOCK AND RESET SIGNALS

clk (Input) -> System clock signal used for synchronous debug operations.

rst_n (Input) -> Active-low reset signal used to initialize the debug unit and return it to a known safe state.

2. DEBUG INTERFACE SIGNALS

These signals provide communication between the processor and the external debugger.

- debug_req_i -> Debug request signal from the external debugger. This signal initiates a debug access request.
- debug_gnt_o -> Debug request grant signal. It acknowledges that the processor has accepted the debug request.

- debug_rvalid_o -> Read data valid signal. Indicates that requested debug read data is available.
- debug_addr_i -> Address used for debug read/write access. This determines which register, CSR, or memory location is being accessed.
- debug_we_i -> Debug write enable signal. Used to distinguish between read and write operations.
- debug_wdata_i -> Write data input for debug write operations.
- debug_rdata_o -> Read data output returned to the debugger.

3. DEBUG CONTROL SIGNALS

- debug_halted_o -> Indicates whether the processor is currently in halted (debug) state.
- debug_halt_i -> Request signal to stop processor execution and enter debug mode.
- debug_resume_i -> Request signal to resume normal processor execution after debugging is complete.

These control signals manage transitions between normal execution and debug mode.

4. CORE CONTROL SIGNALS

- settings_o -> Debug configuration output signal. Used for internal debug mode settings and control.
- trap_i -> Exception or trap indication signal. This helps the debug unit detect exceptional conditions.
- exc_cause_i -> Exception cause information. Used to identify the reason for traps or abnormal processor behavior.
- stall_o -> Pipeline stall signal. Used to freeze normal instruction execution during debugging.
- dbg_req_o -> Internal request to enter debug mode.
- dbg_ack_i -> Acknowledgment signal confirming debug mode entry.

These signals ensure smooth coordination between the processor core and the debug controller.

5. REGISTER FILE INTERFACE

The debug unit can directly access the processor register file.

Read Interface:

- regfile_rreq_o -> Register read request
- regfile_raddr_o -> Register read address
- regfile_rdata_i -> Register read data

Write Interface:

- regfile_wreq_o -> Register write request
- regfile_waddr_o -> Register write address
- regfile_wdata_o -> Register write data

This allows developers to inspect and modify register values during debug sessions.

6. CSR INTERFACE

The module also provides access to Control and Status Registers (CSRs).

Signals:

- csr_req_o -> CSR access request
- csr_addr_o -> CSR address
- csr_we_o -> CSR write enable
- csr_wdata_o -> CSR write data

- csr_rdata_i -> CSR read data

This helps in analyzing processor status, interrupts, privilege modes, and exception handling.

7. PROGRAM COUNTER SIGNALS

pc_if_i -> Address of the next instruction to be fetched.

pc_id_i -> Address of the current instruction being decoded.

These signals help the debugger track program flow and execution position.

8. PIPELINE AND STATUS SIGNALS

instr_valid_id_i -> Indicates whether the current instruction in the decode stage is valid.

sleeping_i -> Indicates whether the processor is in low-power or sleep mode.

These signals provide additional visibility into pipeline activity and processor state.

9. JUMP CONTROL SIGNALS

jump_req_o -> Request signal for forced program counter redirection.

jump_addr_o -> Target address for jump operation.

This is useful for redirecting execution during debugging or exception recovery.

WORKING PRINCIPLE

Step 1: Debug Request

When the external debugger sends debug_req_i = 1, the debug unit begins the debug access process.

Step 2: Halt Processor

If required, the debug unit activates debug_halt_i, which stops normal instruction execution and places the processor into debug mode. The pipeline is stalled using stall_o to ensure a safe halt state.

Step 3: Access Internal State

Once halted, the debugger can:

- Read/write general-purpose registers
- Access CSRs
- Monitor PC values
- Check trap and exception status

This provides complete internal visibility.

Step 4: Resume Execution

After debugging is complete, the debugger sends debug_resume_i, which releases the stall and allows the processor to continue normal execution. This ensures seamless transition back to program execution.

KEY DESIGN ADVANTAGES

- Full processor state visibility
- Safe halt and resume functionality
- Direct register file and CSR access

- Exception and trap monitoring
- Program flow tracking using PC signals
- Pipeline control during debugging
- Strong support for verification and validation

SUMMARY

The zeroriscy_debug_unit is an essential module for processor debugging, validation, and verification.
It provides:

- Controlled halt and resume operations
- Direct access to registers and CSRs
- Pipeline stalling for safe debug entry
- Exception and trap analysis
- Program counter monitoring
- External debugger integration

This module significantly improves development efficiency by enabling complete control and observation of the Zero-Riscy processor during execution, making it indispensable for both hardware verification and software debugging.